Power Of Two

Question

Given an integer n, return true if it is a power of two. Otherwise, return false.

An integer n is a power of two, if there exists an integer x such that n == 2^x.

Example 1:

- Input: n = 1
- Output: true
- Explanation: 2^0 = 1

Constraints

- -2^{31} <= n <= 2^{31} - 1

Solution

so for this question here i do it by using recursion..

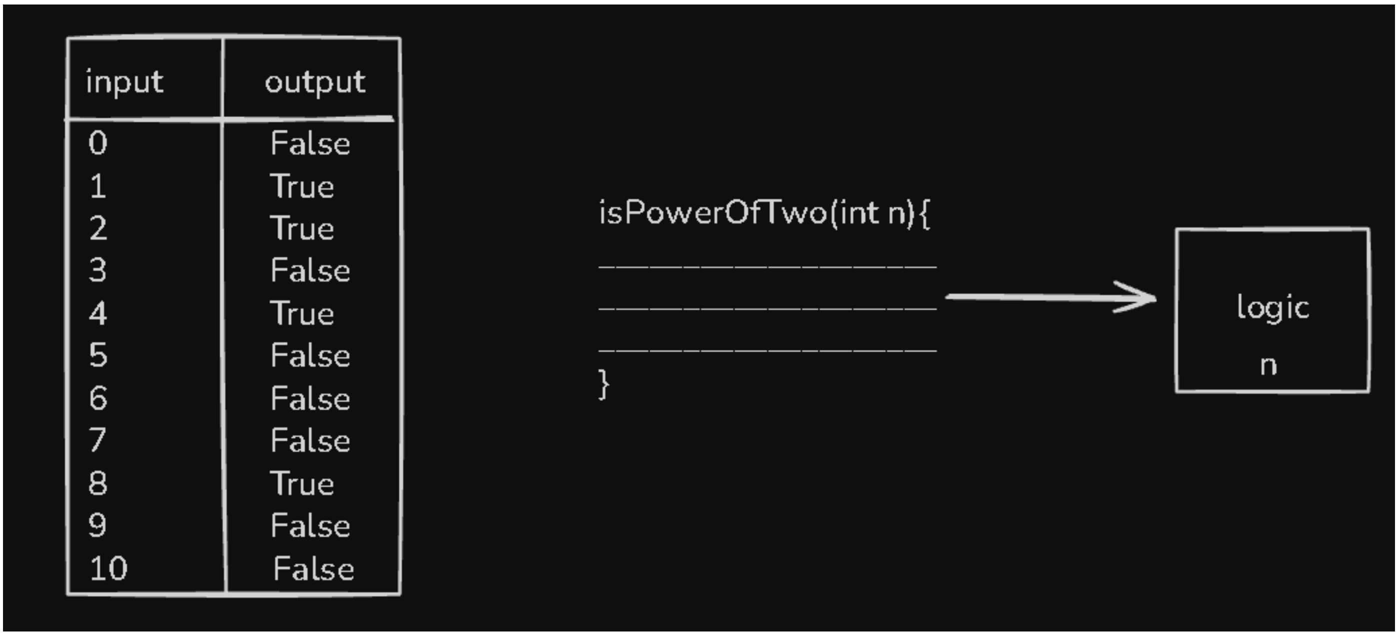
Code

```
class Solution {
    public boolean helper(int num) {
        if (num == 0 ){
            return false;
        }
        if (num == 1) {
            return true;
        } else {
            if (num % 2 != 0) {
                return false;
            }
            return helper(num / 2);
        }

    }

    public boolean isPowerOfTwo(int n) {
```

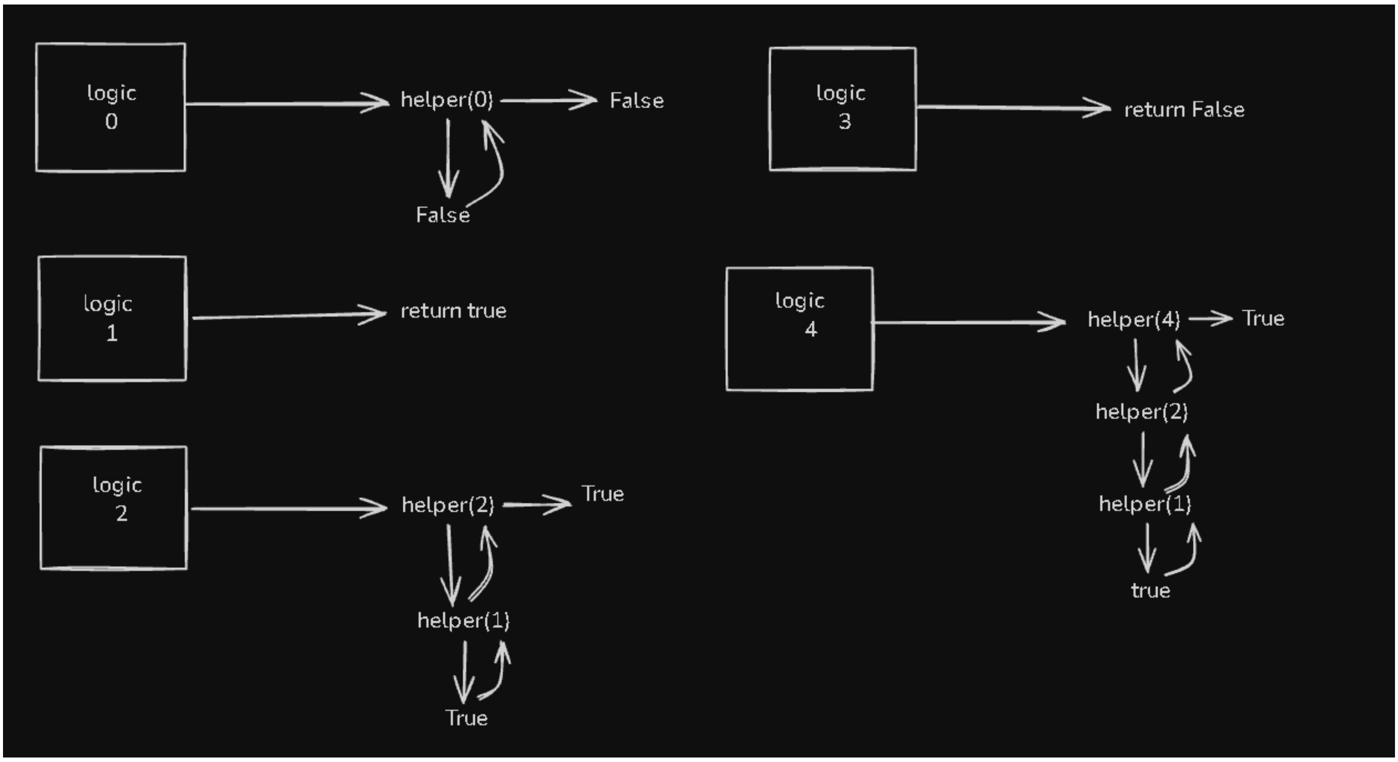
```
        if (n == 1) {
            return true;
        }
        if (n % 2 != 0) {
            return false;
        } else {
            return helper(n);
        }

    }
}
```

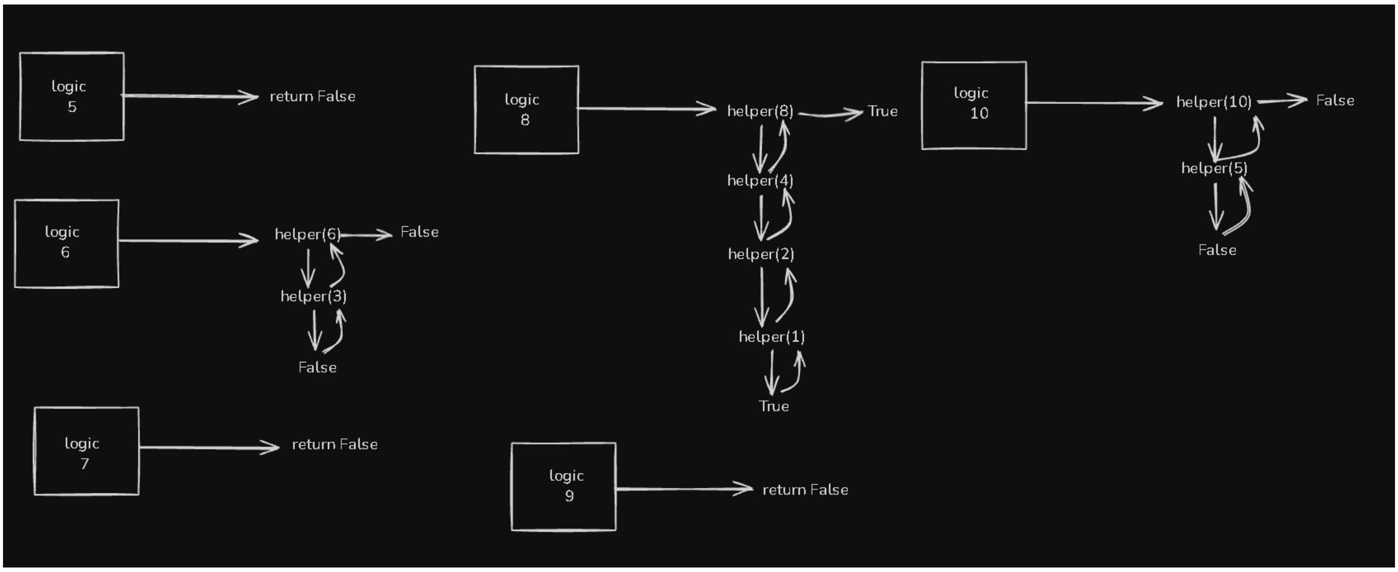
dry run of above code

let run this code for 0 to 10 and examine the outputs.



input	output
0	False
1	True
2	True
3	False
4	True
5	False
6	False
7	False
8	True
9	False
10	False





Close

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

time complexity = $O(log(n))$

space complexity = $O(1)$